WAREHOUSE MANAGEMENT SYSTEM

SDLC Documentation

SRS

Field	Details
Project Name	Warehouse Management System
Technology	Java Spring Boot
Prepared By	Team Mavericks

SRS (Software Requirement Specification)

This section defines the functional and non-functional requirements of the Warehouse Management System in technical terms for developers, architects, and testers.

2.1 Functional Requirements

FR-ID	Module	Description
FR-01	Item Management	System shall allow users to create a new warehouse item with name, SKU, category, and quantity.
FR-02	Item Management	System shall allow retrieval of a single item or all items with filtering by category or SKU.
FR-03	Item Management	System shall allow updating item details including quantity, location, and status.
FR-04	Item Management	System shall allow deletion of an item record with appropriate authorization.
FR-05	Shipping In	System shall record incoming shipments including shipper ID, items, quantities, and timestamp.
FR-06	Shipping In	System shall automatically update stock levels upon recording a shipping-in event.
FR-07	Shipping Out	System shall record outbound shipments with destination, shipper, items, and quantities.
FR-08	Shipping Out	System shall deduct item quantities from stock upon recording a shipping-out event.

FR-09	Shipping Out	System shall prevent shipping out if stock quantity is insufficient and return a validation error.
FR-10	Shipper Mgmt	System shall allow CRUD operations for shipper profiles (name, contact, license, status).
FR-11	Stock Accuracy	System shall provide a real-time stock summary endpoint showing current quantities per item.
FR-12	Reporting	System shall provide shipping history logs filterable by date range, shipper, or item.

2.2 Non-Functional Requirements

Category	Requirement	Target
Performance	API response time for standard CRUD operations	< 300ms
Scalability	Concurrent user support without degradation	Up to 500 concurrent users
Availability	System uptime (excluding planned maintenance)	99.9%
Security	API authentication and authorization	JWT-based Auth (Bearer Token)
Data Integrity	Transactional consistency for stock updates	ACID compliance via PostgreSQL

Maintainability	Code modularity and documentation	Layered architecture (Controller/Service/Repo)
Reliability	Graceful error handling with descriptive messages	HTTP standard error codes + body

2.3 Use Cases

Use Case 1: Record Shipping In

- **Actor:** Warehouse Staff
- **Precondition:** Item and shipper exist in the system
- **Flow:** Submit shipping-in request → system validates data → stock is increased → confirmation returned
- **Postcondition:** Inventory updated and inbound shipment recorded
- **Exception:** 404 if item/shipper not found, 400 for invalid request

Use Case 2: Record Shipping Out

- **Actor:** Warehouse Staff
- **Precondition:** Sufficient stock available
- **Flow:** Submit shipping-out request → system validates stock → stock is reduced → confirmation returned
- **Postcondition:** Inventory updated and outbound shipment recorded
- **Exception:** 409 if stock is insufficient

WAREHOUSE MANAGEMENT SYSTEM

SDLC Documentation

BRD

Field	Details
Project Name	Warehouse Management System
Technology	Java Spring Boot
Prepared By	Team Mavericks

BRD (Business Requirement Document)

This document captures the business objectives, problem statements, stakeholders, and scope for the Warehouse Management System (WMS).

1.1 Executive Summary

The proposed Warehouse Management System (WMS) is a centralized, digital solution designed to optimize and automate warehouse operations. By replacing legacy manual processes, the WMS will provide real-time visibility into inventory, streamline shipping processes, and centralize item management. This digital transformation aims to reduce operational costs, minimize stock discrepancies, and significantly improve overall accountability.

1.2 Business Objective

Warehouse operations often depend on manual methods to track inventory, manage items, and coordinate shipping, which leads to errors, delays, and inaccurate stock data. This Warehouse Management System provides a centralized, API-based solution using Java Spring Boot and PostgreSQL to automate warehouse processes and enable real-time visibility and efficiency.

1.3 Current Situation & Problem Statement

The existing manual warehouse processes suffer from the following critical issues:

- Inventory is tracked manually, causing stock errors and delays.
- Shipping in and out is not monitored in real time, leading to confusion.
- Shipper coordination is scattered and lacks accountability.
- Manual item updates are slow, error-prone, and hard to track.
- Poor data management makes scaling and reporting difficult.

1.4 Business Goals

The following measurable goals define the success of the WMS implementation:

#	Goal	Target Metric
G-1	Automate inventory tracking	100% digital records

G-2	Real-time shipping visibility	< 5 sec data refreshes
G-3	Centralize CRUD for item management	Single API gateway
G-4	Reduce stock discrepancy	< 1% error rate
G-5	Improve shipper coordination	Full shipper traceability

1.5 Stakeholders

Stakeholder	Role	Responsibility
Warehouse Owner/Manager	Business Owner	Define requirements, approve scope
Warehouse Staff	End User	Manage inventory, shipping in/out
Shippers/Logistics Partners	External User	Coordinate shipment records
Development Team	Engineering	Design, build, and deploy the system
QA Team	Testing	Validate functional and performance requirements

1.6 Project Scope

In Scope:

- Item management – Create, Read, Update, Delete (CRUD) operations for warehouse items.
- Shipping In – Record and track all goods arriving at the warehouse.
- Shipping Out – Record and track all goods dispatched from the warehouse.

- Shipper management – CRUD operations for shipper profiles and coordination.
- Stock accuracy – Real-time stock level computation based on shipping activity.
- RESTful API – Spring Boot API as the backend service layer.
- Database – PostgreSQL as the primary data store for all warehouse data.

Out of Scope (Phase 1):

- Mobile application or frontend UI (API only).
- Payment or invoice processing.
- Third-party ERP or accounting system integrations.
- Multi-warehouse support.

1.7 Success Criteria and ROI

- **Success Criteria:** 100% digital transition for inventory tracking within 30 days of launch; elimination of manual logging; API response times stabilized under target metrics.
- **ROI:** Projected to achieve positive ROI within 12-18 months by reducing missing stock by 99% and significantly reducing labor hours spent on manual audits and shipper coordination.

WAREHOUSE MANAGEMENT SYSTEM

SDLC Documentation

HLD

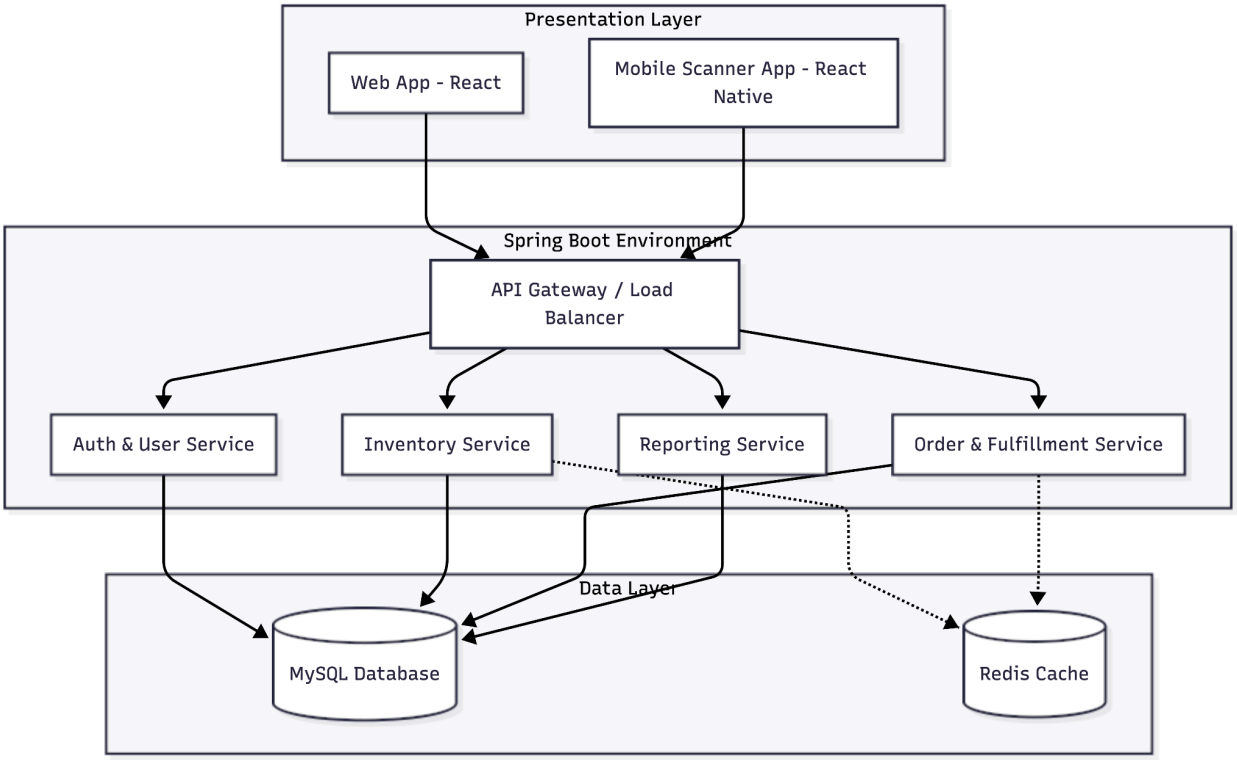
Field	Details
Project Name	Warehouse Management System
Technology	Java Spring Boot
Prepared By	Team Mavericks

HLD – High Level Design

This section describes the overall architecture, technology stack, system components, and data model of the Warehouse Management System(WMS) from a high-level perspective.

3.1 System Architecture

The WMS follows a classic three-tier RESTful architecture:



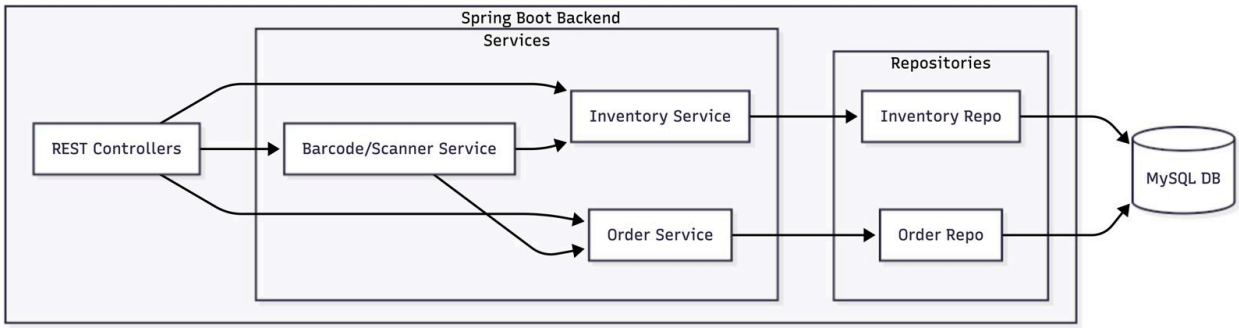
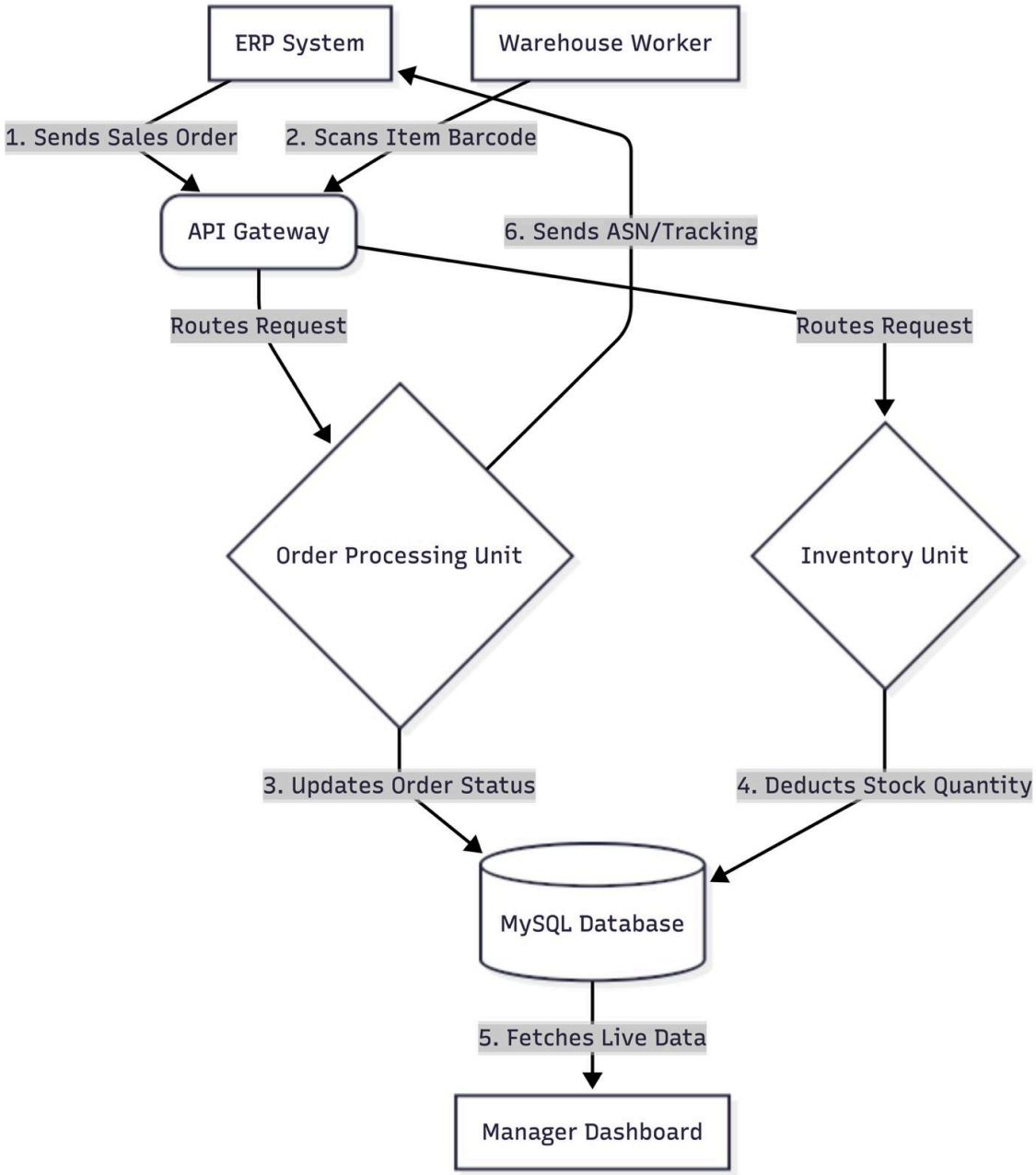
3.2 Technology Stack

Layer	Technology	Justification
Backend Framework	Java Spring Boot	Production-grade REST API, embedded Tomcat
Language	Java 17	Stability, strong typing, enterprise ecosystem
Database	MySQL	ACID transactions, relational integrity, JSON support
ORM	Spring Data JPA / Hibernate	Simplified DB operations, entity mapping
Security	Spring Security + JWT	Stateless auth, role-based access control
Build Tool	Maven / Gradle	Dependency management and build automation
API Testing	Postman / Swagger UI	API documentation and manual testing
Version Control	Git / GitHub	Source code management and collaboration
Deployment	Docker + Cloud (AWS)	Containerized, scalable deployment

3.3 System Components

Component	Responsibility
REST Controllers	Handle HTTP requests, validate inputs, return structured JSON responses.
Service Layer	Business logic: stock calculation, validation rules, transaction management.
Repository Layer	Database access via Spring Data JPA; custom queries for reports and filtering.
Entity Models	JPA entity classes mapping to PostgreSQL tables (Item, Shipper, ShippingIn, ShippingOut).
DTO Layer	Data Transfer Objects for request/response decoupling from entity models.
Exception Handler	Global @ControllerAdvice to return consistent error responses with HTTP codes.
Security Module	JWT token validation, filter chain, and role-based endpoint protection.

3.4 Component Diagram:



3.5 Dataflow Diagram: